



National University of Science and Technology Politehnica Bucharest
Faculty of Automatic Control and Computers

MASTER THESIS

Data-Driven Urban Housing Analysis for Sustainable City Living

Student
Merve Pakcan

Advisor
Prof. dr. ing. Monica Drăgoicea

Bucharest, 2026

Contents

- 1 Research Report 3. Advanced Visual Analytics and Tree-Based Machine Learning Methods for Urban Housing Analysis 1**
 - 1.1 Problem Framing and Decision Context 1
 - 1.2 Data Overview 2
 - 1.3 Tree-Based Machine Learning Models 2
 - 1.3.1 Decision Tree Algorithms 3
 - Splitting Criteria 3
 - Tree Pruning 3
 - Regression Trees vs. Classification Trees 3
 - 1.3.2 Ensemble models 4
 - Bagging Versus Boosting 4
 - 1.4 Modeling Workflow and Evaluation 6
 - 1.4.1 Decision Tree Model 6
 - Regression Tree Interpretation and Visualization 6
 - Default Decision Tree 11
 - Tuned Decision Tree 13
 - 1.4.2 Random Forest Model 14
 - Default Random Forest 14
 - Tuned Random Forest 15
 - 1.4.3 Gradient Boosting Models 16
 - Default Gradient Boosting 17
 - Tuned Gradient Boosting 18
 - 1.5 Model Comparison 19
 - 1.6 Conclusion 20
- Bibliography 22**

1 Research Report 3. Advanced Visual Analytics and Tree-Based Machine Learning Methods for Urban Housing Analysis

1.1 Problem Framing and Decision Context

EARLIER stages of this research addressed sustainability challenges from a broad perspective and subsequently narrowed the analytical focus to urban housing markets, recognizing housing as a key domain where economic conditions, social equity, and spatial dynamics intersect. In many urban areas, rising housing prices, increasing rental pressure, and pronounced regional disparities have intensified affordability concerns, directly shaping residential choices and market behavior.

Building on the exploratory and predictive analyses developed in previous chapters, this chapter does not introduce a new problem but rather deepens the analytical treatment of the existing housing affordability challenge. The focus shifts from identifying patterns and model performance toward understanding how these patterns emerge and how they can be systematically interpreted.

In complex and heterogeneous housing markets, relying solely on numerical performance metrics is often insufficient. While predictive accuracy is important, it provides limited insight into the structural drivers and decision rules underlying market dynamics. Advanced visual analytics, combined with interpretable tree-based machine learning models, offer a complementary approach by enabling interactive exploration, transparent rule extraction, and comparison across regions and time.

Accordingly, this chapter concentrates on the use of visual analytics and tree-based modeling, specifically decision trees and ensemble methods, to examine affordability-driven patterns and regional dynamics in urban housing markets. By emphasizing interpretability and visual explanation alongside predictive modeling, the analysis aims to provide a clearer and more structured understanding of housing market behavior.

The analysis is structured around the following research questions:

- Which affordability and competitiveness indicators most strongly influence housing prices across U.S. regions?
- How effectively can tree-based machine learning models explain and predict housing price variation compared to traditional linear approaches?
- What decision-relevant patterns emerge from visual analytics and rule-based interpretation of tree-based models?

1.2 Data Overview

The analysis uses a consolidated panel dataset derived from Zillow Research, a public platform providing regularly updated indicators of the U.S. housing market. The dataset spans March 2018 to February 2025 and captures monthly observations across U.S. regions (e.g., metro/city level), enabling both temporal and regional comparisons. Following the data integration process described in earlier work, seven Zillow series were reshaped to a consistent format and merged using common geographic identifiers (RegionID, RegionName, RegionType, StateName) and a monthly Date field.

After quality control and missing-value handling, the working dataset contains approximately 9,893 observations and 12 variables. The target variable for modeling is housing price, represented by Median Sale Price, while explanatory variables capture affordability, rental pressure, and market competitiveness. Key predictors include Median Rent (ZORI), Years to Save (affordability), Market Heat Index, Percentage Sold Above List, Sale-to-List Ratio, and New Construction Median Sale Price where available.

This chapter focuses on developing interpretable tree-based machine learning models to analyze housing price dynamics.

1.3 Tree-Based Machine Learning Models

Urban housing markets exhibit strong nonlinearity and heterogeneity across regions. Housing prices are shaped by the combined effects of affordability, rental pressure, market competitiveness, and local economic conditions. Capturing these complex relationships is challenging for traditional statistical models, which often rely on restrictive assumptions regarding functional form and variable interactions. This limitation of parametric hedonic models in capturing nonlinear and interaction effects has also been emphasized in recent real estate literature, which highlights the restricted inferential capacity of traditional approaches when faced with complex market dynamics [10].

Tree-based machine learning methods offer a flexible and interpretable framework for analyzing such complex market structures. By recursively partitioning the input space using simple decision rules, decision trees can capture nonlinear effects and interaction patterns present in the data. This makes tree-based models particularly suitable for exploratory housing market analysis, where understanding the drivers of price variation is as important as prediction accuracy. Beyond single-tree models, this family also includes ensemble methods such as random forests and gradient boosting, which combine multiple decision trees to improve predictive performance and robustness.

Within this modeling family, decision trees constitute the core analytical component. A decision tree organizes the data into a hierarchical structure, where each split is based on a single explanatory variable and each terminal node represents observations with similar characteristics. As nested rule-based structures, decision trees translate complex data patterns into transparent if-else decision paths, thereby enabling straightforward interpretation of how key variables influence housing prices[10].

These models will later be implemented and compared in SAS Viya, which provides an integrated workflow for model development, tuning and evaluation across single trees, random forests and gradient boosting methods.

1.3.1 Decision Tree Algorithms

Splitting Criteria

Decision trees construct a hierarchical partition of the predictor space by recursively selecting splits that minimize within-node variability of the target variable. Since housing prices are continuous outcomes, this study uses regression trees, where split selection is based on variance reduction, also referred to as the residual sum of squares (RSS) or ANOVA criterion [22].

Since the response variable is continuous, classification-based and statistical test-based splitting criteria such as Gini index, entropy, chi-square, or CHAID were not applicable and therefore not used. In SAS Viya, regression trees rely on binary splits and evaluate candidate thresholds across continuous predictors using this variance-based criterion, resulting in progressively more homogeneous price segments.

The first split occurs on YearsToSave, indicating that affordability-related factors explain a substantial share of the observed variation in housing prices.

Tree Pruning

Classical post-pruning techniques, such as cost-complexity pruning, were not explicitly applied in this study. Instead, tree complexity was controlled through pre-pruning constraints implemented via the decision tree hyperparameters available in SAS Viya.

Model growth was regulated using parameters such as maximum tree depth, minimum leaf size, and interval binning, which limit overly specific splits during training. These constraints helped reduce sensitivity to noise and supported more stable generalization.

As the regression tree was designed to serve as an interpretable baseline model, pre-pruning was preferred in order to preserve transparent decision rules and coherent segmentation structures. This modeling choice is consistent with prior housing market studies that emphasize the use of decision trees as interpretable tools for uncovering heterogeneous pricing structures and attribute-based segmentation [4]. Additional regularization and performance gains were subsequently obtained through ensemble methods.

Regression Trees vs. Classification Trees

Decision trees can be applied to both classification and regression problems, depending on the nature of the response variable (Figure 1.1). Classification trees are used when the outcome is categorical, whereas regression trees are designed for continuous response variables. In the context of housing market analysis, where prices are continuous outcomes, this study focuses on regression trees. Regression trees partition the predictor space using threshold-based splits, with each terminal node associated with a representative price level. This structure supports both prediction and interpretable analysis of housing price variation under different market conditions [22].

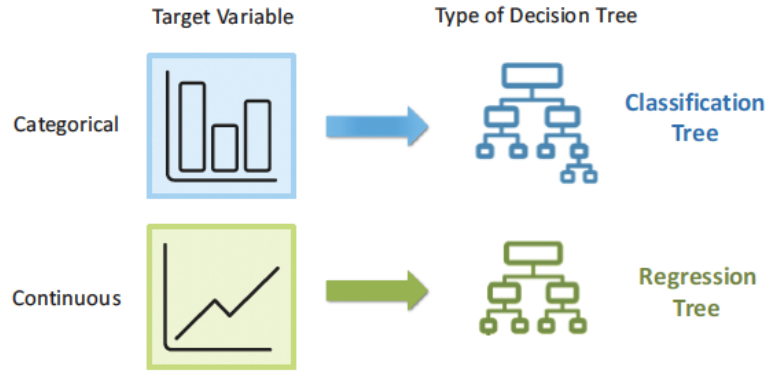


Figure 1.1: Types of Decision Trees, source: [22]

Given the continuous nature of housing prices, regression trees constitute the baseline model for this study and will serve as the foundation for subsequent ensemble methods.

1.3.2 Ensemble models

Bagging Versus Boosting

Ensemble learning methods enhance predictive performance by aggregating the outputs of multiple decision trees. Within this framework, bagging and boosting represent two distinct ensemble strategies that differ in their learning design and approaches to error reduction. Bagging trains multiple decision trees independently using bootstrap samples. The predictions of these trees are then combined through simple averaging. This strategy primarily targets variance reduction and leads to more stable model predictions. Boosting, in contrast, follows a sequential learning process. Each new tree is trained to correct the errors made by previous models. As a result, boosting focuses on reducing bias and often achieves higher predictive accuracy[6].

Given the heterogeneous structure of housing prices, both ensemble strategies are considered as candidate modeling approaches in this study. Random Forest and Gradient Boosting are selected as representative implementations of bagging and boosting, respectively. Comparing these models allows an assessment of whether variance-focused aggregation or bias-reducing sequential learning is more effective for housing price prediction.

Table 1.1 provides a comparative summary of the conceptual differences between bagging and boosting, including sampling goals, learning structures, and representative algorithms.

	Bagging	Boosting
Sampling	Randomly chosen unweighted cases	Higher weight for misclassified cases
Goal	Minimize variance	Increase accuracy (reduce bias)
Combining models	Weighted average	Weighted majority weight
Learning structure	Parallel trees	Sequential trees
Example algorithms	Random Forest	Gradient Boosting

Table 1.1: Comparison of Bagging and Boosting methods (adapted from [22])

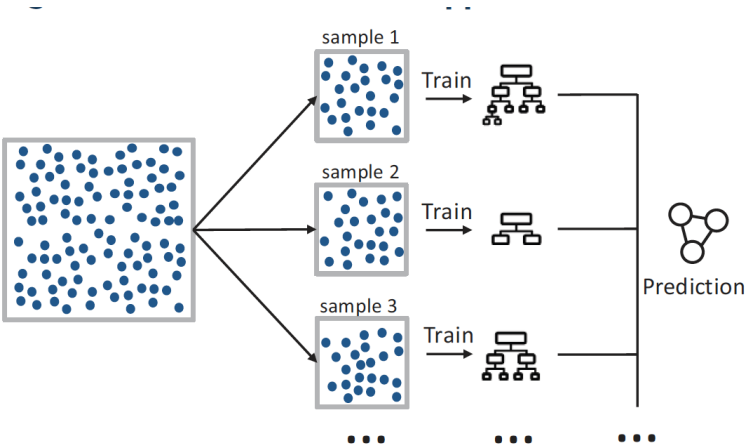


Figure 1.2: Parallel Ensemble Approach, source: [22]

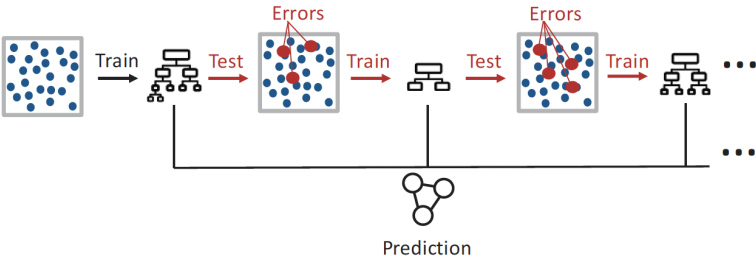


Figure 1.3: Sequential Ensemble Approach, source: [22]

Beyond these operational aspects, the structural divergence between the two methods is illustrated in Figures 1.2 and 1.3. As shown in Figure 1.2, bagging constructs multiple decision trees in parallel and aggregates their predictions, leading to reduced variance and more stable outcomes. Figure 1.3 highlights the sequential nature of boosting, where each tree iteratively refines the model by focusing on previously mispredicted observations. This sequential learning mechanism is particularly effective for capturing complex, nonlinear relationships commonly observed in urban housing markets.

From a bias–variance perspective, Random Forest reflects a variance-reducing ensemble strategy, while Gradient Boosting follows a bias-reducing approach. Evaluating both models enables a systematic assessment of which mechanism better captures housing price dynamics in a heterogeneous market environment.

1.4 Modeling Workflow and Evaluation

The modeling workflow was implemented in SAS Viya using the Build Models environment to formalize the training and evaluation of the regression tree, random forest, and gradient boosting models. All models were trained using a 60/20/20 train-validation-test split, enabling out-of-sample performance assessment and reducing the risk of overfitting. The same data partition and evaluation strategy was maintained across all models to ensure comparability of results.

1.4.1 Decision Tree Model

Regression Tree Interpretation and Visualization

The regression decision tree served as the baseline model for predicting the continuous target variable, Median Sale Price. This model was selected due to its ability to capture nonlinear interactions while maintaining a high level of interpretability through a rule-based structure. No hyperparameter tuning was applied at this stage, as interpretability and rule extraction were prioritized over predictive optimization. For the regression decision tree, the Explore and Visualize environment was additionally used to inspect splitting structures, variable importance, node rules, and diagnostic outputs before formalizing the model as a pipeline (Figure 1.4). This visual exploration step provided interpretability insights that informed the baseline configuration and supported the explanatory goals of the analysis. Establishing the regression tree as a baseline enables the subsequent ensemble models to be evaluated comparatively in terms of both predictive accuracy and structural complexity.

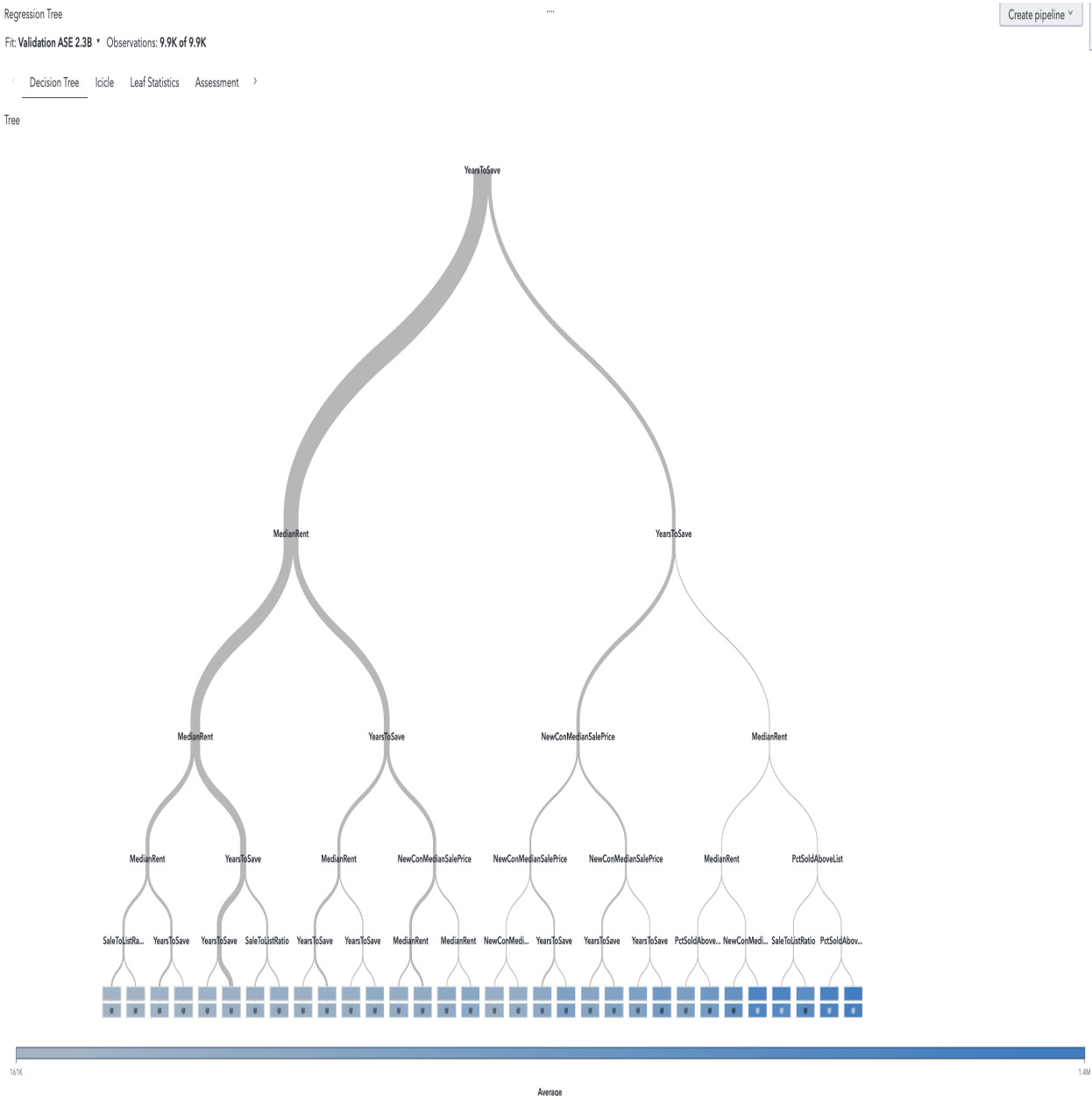


Figure 1.4: Regression Tree

The root node represents the entire dataset, and the first split is performed on the `YearsToSave` variable. This indicates that `YearsToSave` provides the greatest reduction in prediction error at the top level of the tree and is therefore identified as the most influential variable in the initial partitioning of the data.

As the tree grows downward, additional splits are performed using variables such as `MedianRent`, `NewConMedianSalePrice`, and `MarketHeatIndex`. The repeated use of these variables at higher levels of the tree suggests that they play a significant role in explaining variations in median sale prices. The thickness of the branches reflects the number of observations flowing through each split, with thicker branches corresponding to larger data subsets.

The leaf nodes at the bottom of the tree represent final market segments consisting of observations with similar characteristics. For each leaf node, the model produces a single predicted value corresponding to the average median sale price of that segment. These predicted values are visually encoded using a color gradient, where darker shades indicate higher predicted prices. This visualization highlights how the regression tree partitions the housing market into distinct price segments.

After fitting the model on 9.9K observations, the regression tree yields an ASE of approximately 2.3B. Beyond predictive accuracy, the model is primarily informative due to the interpretability provided by its hierarchical splitting structure.

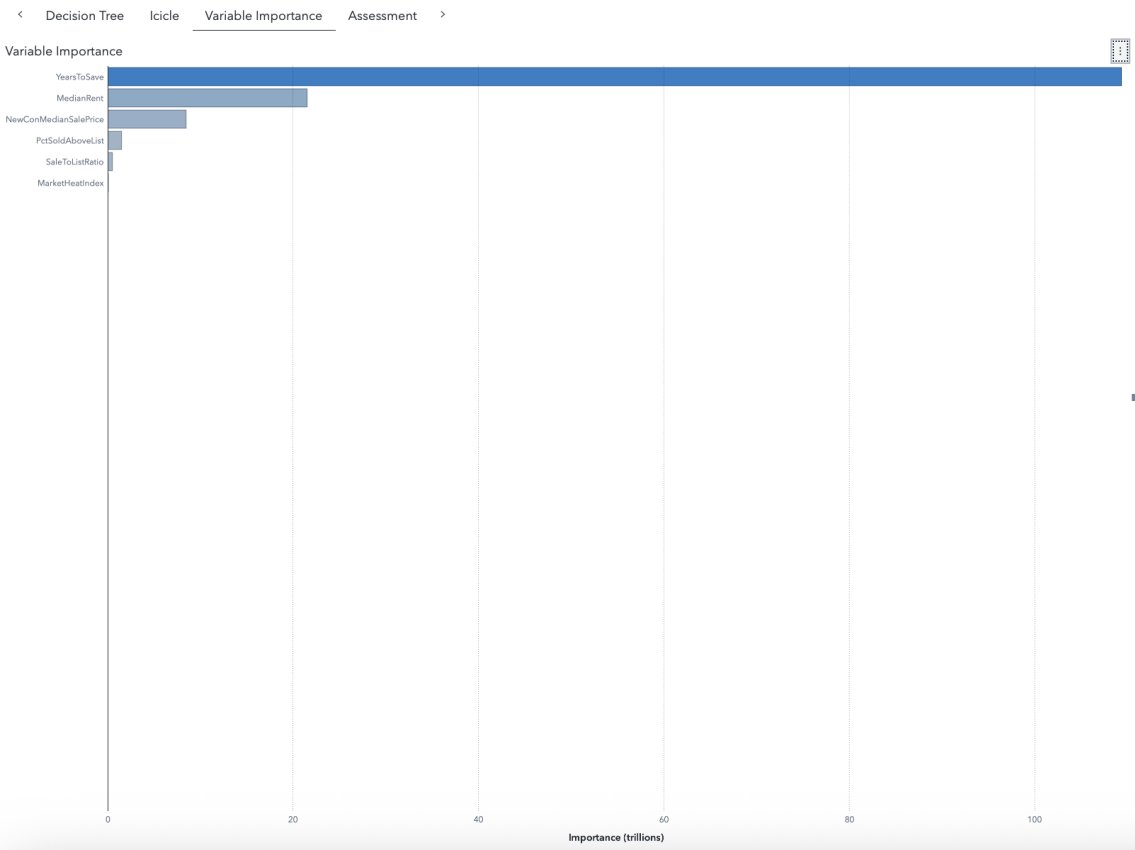
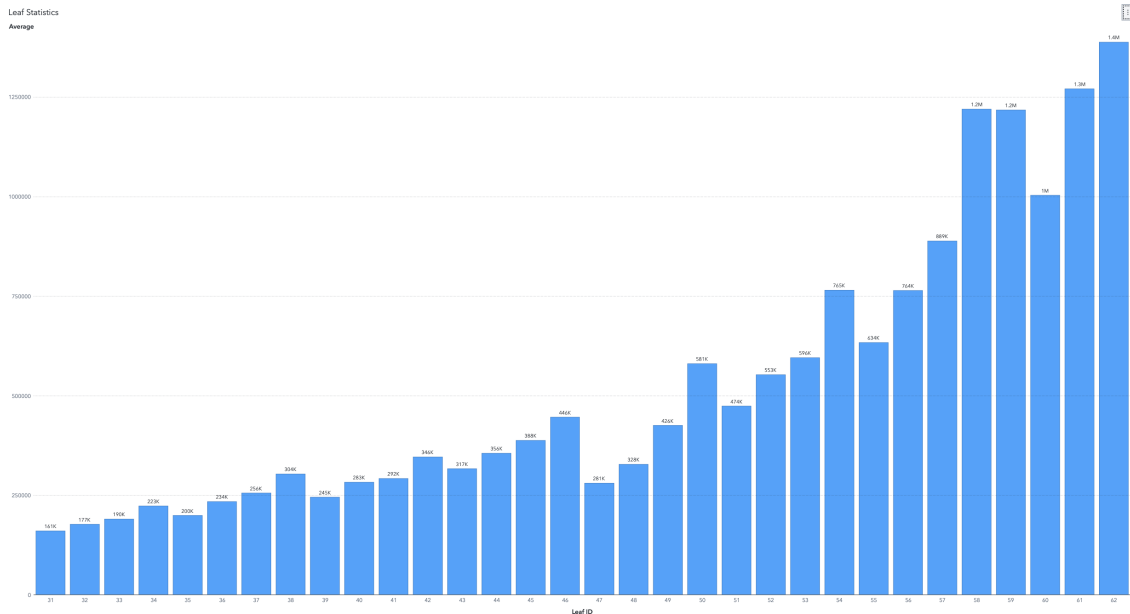


Figure 1.5: Variable Importance

Variable importance results show a dominant contribution from YearsToSave, with MedianRent and NewConMedianSalePrice following at a secondary level (Figure 1.5). Competitive market indicators contribute marginally, suggesting that affordability rather than short-term bidding pressure drives the segmentation learned by the tree model. This pattern will be reassessed using ensemble methods, which may capture more complex interactions.

The icicle visualization provides an alternative representation of the regression tree, illustrating the hierarchical structure of successive splits (Figure 1.6). Large blocks associated with YearsToSave and MedianRent dominate the upper levels of the hierarchy, indicating that affordability and rental pressure variables drive the primary segmentation of markets. Competitive indicators such as SaleToListRatio and PctSoldAboveList appear only in lower layers, refining distinctions within already segmented clusters. This hierarchical pattern aligns with the variable importance results and reinforces the interpretability of the baseline model.



The leaf statistics plot summarizes the final segmentation produced by the regression tree (Figure 1.7). Each leaf corresponds to a group of observations sharing similar market characteristics, and the associated bar reports the average predicted median sale price for that cluster. The ordering of leaf means reveals a clear price hierarchy, with lower-priced clusters on the left and progressively higher-priced clusters on the right. This indicates that the model formed coherent and non-overlapping price segments, validating the segmentation structure learned by the tree.

Node Statistics	Node Rules	Variable Importance	Cost-Complexity Pruning	Assessment	Assessment Statistics	Assessment Summary																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																		</
-----------------	------------	---------------------	-------------------------	------------	-----------------------	--------------------	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	----

Figure 1.8: Node Rules

The Node Rules output summarizes how the regression tree partitions observations based on variable thresholds (Figure 1.8). Each row corresponds to a node and shows the rules that route observations from the root toward terminal leaves. The resulting rule-based segmentation highlights the role of affordability-related variables, particularly YearsToSave and MedianRent, in forming distinct price clusters. Although the input dataset does not contain missing values, SAS indicates “missing” as part of its surrogate-splitting logic, reflecting how the algorithm would handle potential missing paths.

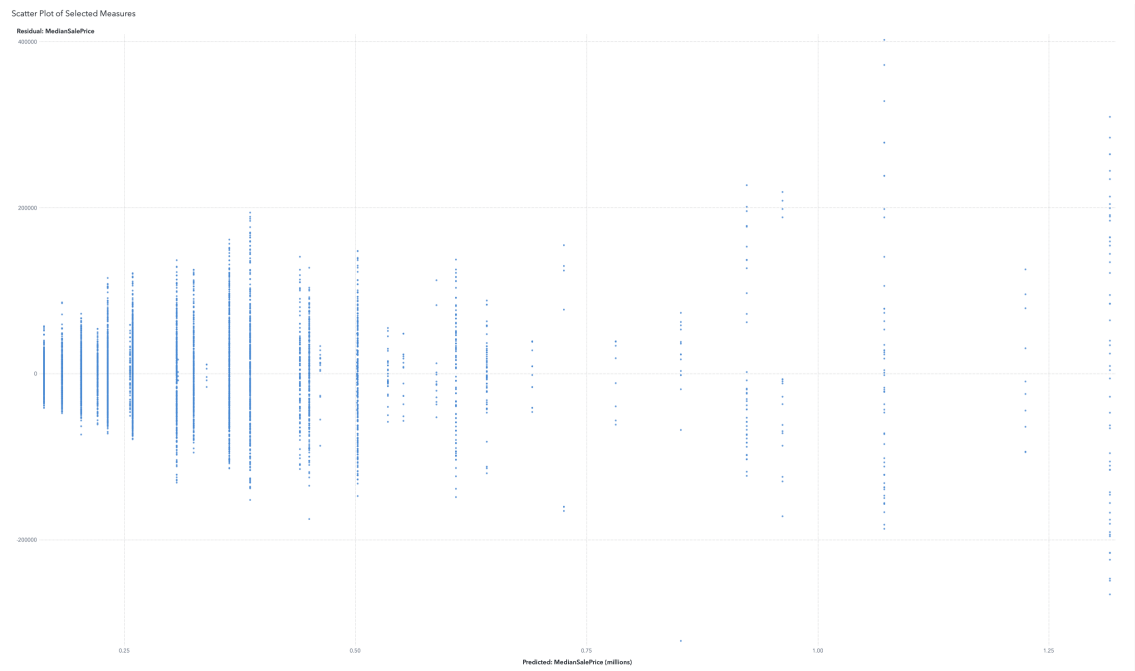


Figure 1.9: Residual Plot

The residual plot (Figure 1.9) shows the typical pattern of a regression tree. Residuals appear in vertical bands because each terminal node produces a constant prediction. The residuals are centered around zero, so the model does not systematically over- or under-predict prices. Residuals spread out at higher price levels, reflecting greater variability in expensive markets. Overall, the tree provides useful segmentation, but the lack of smoothing suggests that ensemble methods may improve performance. This pattern is consistent with the heterogeneity observed in real housing markets, where price dispersion increases in high-demand regions.

Default Decision Tree

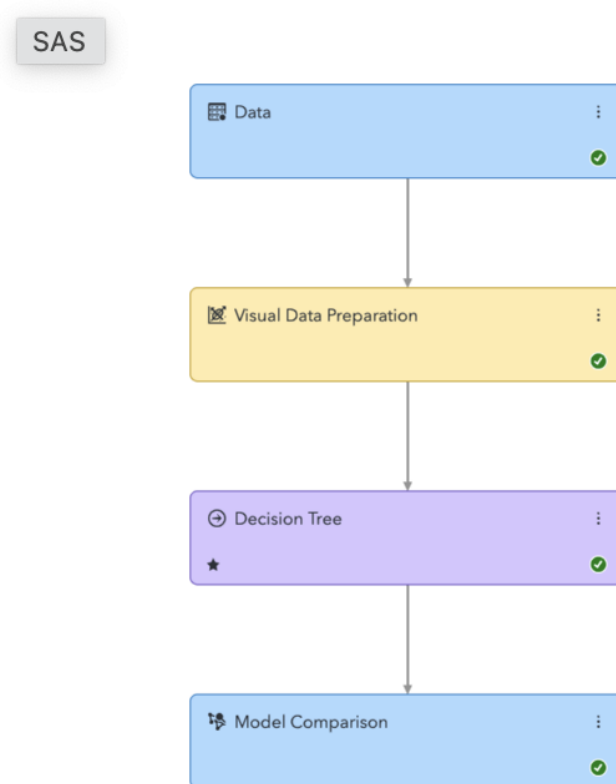


Figure 1.10: Decision Tree Pipeline

Following the exploratory modeling phase, the regression tree was transferred to Model Studio as a pipeline to organize the modeling workflow. The pipeline combines data preparation, decision tree training, and model evaluation within a single structure (Figure 1.10). This setup also makes it easier to extend the analysis with ensemble tree-based models in the next sections, while keeping the evaluation approach consistent and allowing for tuning when required.

Fit Statistics											
Target Name	Data Role	va_d PARTITION	Formatted Partition	Number of Obser...	Average Squared ...	Divisor for ASE	Root Average Squa...	Mean Absolute Er...	Root Mean Absol...	Mean Squared Lo...	Root Mean Squar...
MedianSalePrice	VALIDATE	0	0	1,979	2,310,218,782.9600	1,979	48,064.7353	32,099.7685	179.1641	0.0158	0.1258
MedianSalePrice	TRAIN	1	1	5,935	1,704,884.844	5,935	41,290.2512	29,973.8475	173.1296	0.0147	0.1214
MedianSalePrice	TEST	2	2	1,979	2,336,706,954.3500	1,979	48,339.4968	32,158.7154	179.3285	0.0165	0.1284

Figure 1.11: Decision Tree Fit Statistics

The regression tree exhibits consistent predictive performance across all data partitions, achieving an R^2 of 0.934 on the training set, 0.912 on the validation set, and 0.918 on the test set. RMSE increases from approximately 41,300 dollars (train) to 48,300 dollars (test), indicating that errors remain within a reasonable range given the dispersion of metropolitan housing prices (Figure 1.11). Nevertheless, the presence of residual variance indicates that additional factors or more flexible modeling approaches could further enhance predictive performance. For this reason, the regression tree is treated as a baseline model against which more advanced ensemble methods.

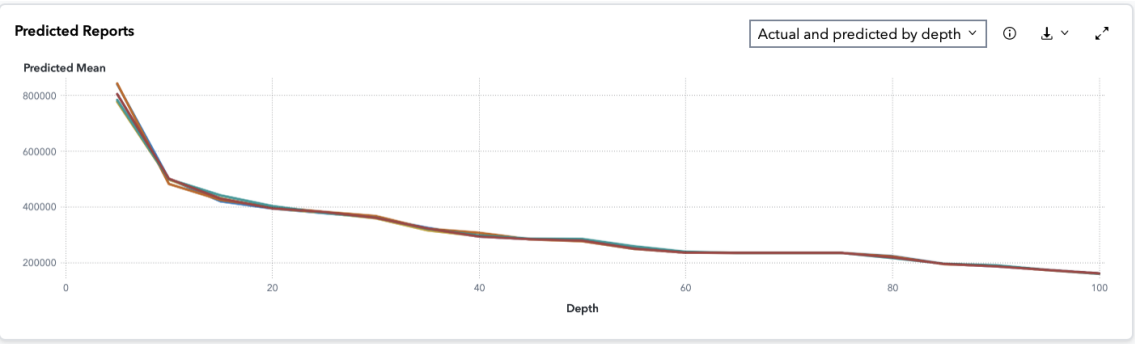


Figure 1.12: Predicted Reports Decision Tree

The predicted reports plot shows that most of the price differentiation is captured within the first few splits of the tree, after which the predicted mean stabilizes (Figure 1.12). The close alignment between training, validation, and test series further indicates stable generalization without pronounced overfitting.

Tuned Decision Tree

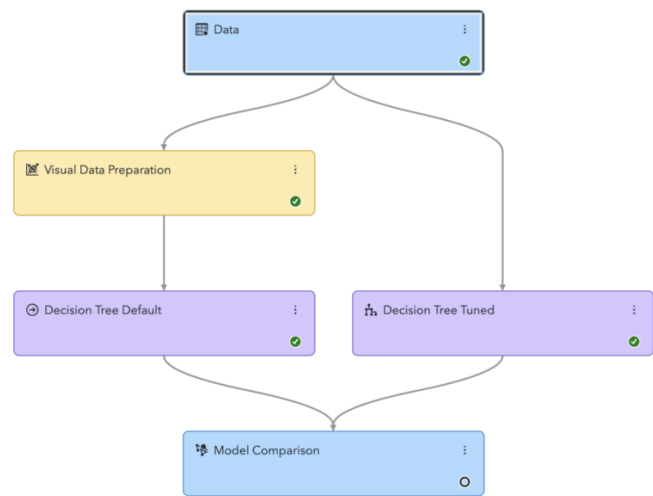


Figure 1.13: Tuned Decision Tree Pipeline

The pipeline was created using the Building Model component in SAS Viya(Figure 1.13). Hyperparameter tuning was applied to the decision tree model to improve predictive performance while preserving its interpretability as a baseline learner. Instead of modifying all available parameters, tuning was limited to a small set of hyperparameters that directly influence model complexity and generalization. The maximum tree depth was increased from 5 to 10 to better capture non-linear relationships in housing price dynamics that were not adequately represented in the default configuration. In parallel, the minimum leaf size was increased from 5 to 30 to limit overly specific terminal nodes and reduce sensitivity to noise. The number of interval bins was increased from 50 to 100 to allow finer splits for continuous variables. Other structural settings, including the variance-based splitting criterion and binary branching, were kept unchanged to preserve interpretability and ensure methodological consistency.

Fit Statistics

Target Name	Data Role	va_d...	Formatted Partition	Number of Observations	Average Squared Error	Divisor for A...	Root Average Squared Er...	Mean Absolute Error	Root Mean Absolute Error	Mean Squared Log...	Root Mean Squa...	R-Square
MedianSalePrice	VALIDATE	0	0	1,979	1,681,508,251.8500	1,979	40,018.8487	25,550.2102	159.8443	0.0100	0.1000	0.9391
MedianSalePrice	TRAIN	1	1	5,935	1,247,636,990.1900	5,935	35,321.9052	22,777.0931	150.9208	0.0079	0.0889	0.9518
MedianSalePrice	TEST	2	2	1,979	1,520,934,919.7100	1,979	38,999.1656	25,416.2729	159.4248	0.0100	0.0999	0.9466

Figure 1.14: Tuned Decision Tree Fit Statistics

Compared to the default decision tree, the tuned model shows a clear improvement in predictive performance across all data partitions. On the test set, the RMSE decreased from about 48.3K to 39.0K, while the R^2 value increased from 0.918 to 0.947, indicating that the tuned tree fits the housing price structure more effectively (Figure 1.14).

Model Comparison

Champion	Name	Algorithm Name	Average Squared Error	Data Role	Number of Observations	Root Average Squared Error	Root Mean Absolute Error	Root Mean Squared Logarithmic Error
★	Decision Tree Tuned	Decision Tree	1,520,934,919.7100	TEST	1,979	38,999.1656	159.4248	0.0999
	Decision Tree Default	Decision Tree	2,336,706,954.3500	TEST	1,979	48,339.4968	179.3285	0.1284

Figure 1.15: Decision Tree Model Comparison

Comparison between default and tuned decision tree models shows that the tuned configuration achieves lower error metrics and a higher R^2 value on the test set. This improvement is mainly driven by increased model capacity through deeper splits, while larger leaf sizes and pruning effectively control model complexity. As a result, the tuned tree reduces underfitting without introducing overfitting and provides more stable generalization than the default model(Figure 1.15).

1.4.2 Random Forest Model

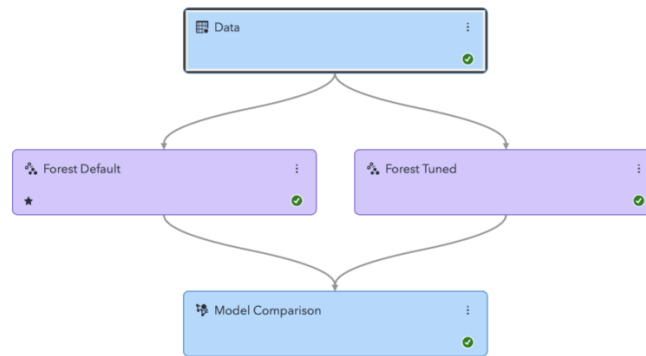


Figure 1.16: Random Forest Pipeline

Random forest modeling was implemented using two configurations: a default ensemble and a tuned ensemble. As a bagging-based method, random forests combine bootstrap aggregation with random feature selection, reducing variance relative to the single regression tree. The default configuration serves as the baseline ensemble, while the tuned model adjusts key hyperparameters to improve predictive accuracy and control overfitting. Both models were trained and evaluated within the same pipeline, ensuring methodological consistency and allowing direct comparison under identical train-validation-test partitions (Figure 1.16).

Default Random Forest

As a baseline ensemble model, a default Random Forest was trained using the SAS Viya Forest node without hyperparameter tuning. This baseline configuration represents a standard bagging-based ensemble and provides a benchmark for evaluating the incremental value of tuning and boosting in subsequent models.

In the default configuration, the Random Forest model was trained with 100 trees, a maximum depth of 20, a minimum leaf size of 5, and approximately three variables considered at each split (following the common heuristic based on the number of predictors). The bootstrap sampling rate was set to 0.60. This configuration reflects a relatively low-bias and higher-variance design in which moderately deep individual trees are aggregated to enhance predictive accuracy.

Fit Statistics											
Target Name	Data Role	va_d_PARTITIO...	Formatted Partition	Number of Obser...	Average Squared ...	Divisor for ASE	Root Average Squa...	Mean Absolute Er...	Root Mean Absol...	Mean Squared Lo...	Root Mean Squar...
MedianSalePrice	VALIDATE	0	0	1,979	768,472,190.2830	1,979	27,721.3310	17,165.9310	131.0188	0.0047	0.0684
MedianSalePrice	TRAIN	1	1	5,935	427,948,505.9970	5,935	20,686.9163	13,005.9719	114.0437	0.0028	0.0533
MedianSalePrice	TEST	2	2	1,979	833,417,963.7540	1,979	28,868.9793	17,537.6007	132.4296	0.0050	0.0704

Figure 1.17: Default Random Forest Fit Statistics

The default Random Forest exhibited strong predictive performance across partitions, achieving an R^2 of 0.983 on the training set, 0.971 on the test set, and 0.971 on the validation set (Figure 1.17). The RMSE reduced substantially relative to the regression tree, decreasing from approximately 48K dollars on the test split of the regression tree to around 28K dollars on the test split of the Random Forest. This improvement indicates that the ensemble effectively captured more of the non-linear structure characterizing metropolitan housing prices.

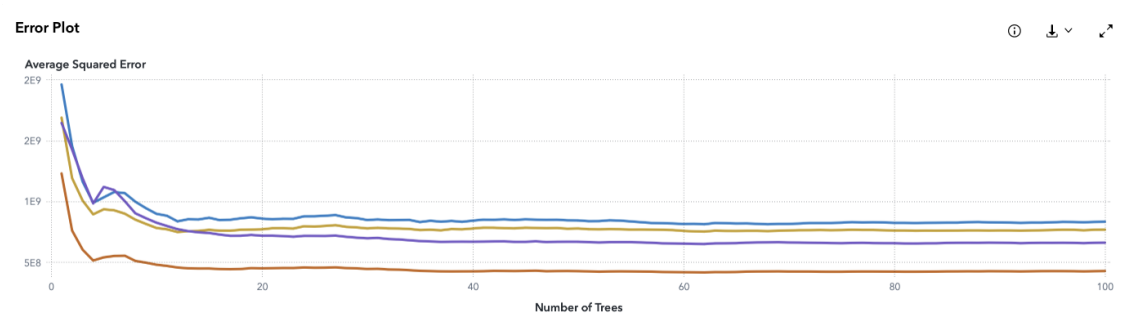


Figure 1.18: Default Random Forest Error Plot

Error decreases rapidly with the first trees and then stabilizes, with training, validation, and test curves remaining closely aligned (Figure 1.18). This indicates stable generalization and diminishing returns beyond approximately 30 trees.

Tuned Random Forest

In the tuned configuration, the number of trees was increased to 300 and the maximum depth to 25, while the minimum leaf size was enlarged to 10. The number of variables considered per split remained fixed at three, and the bootstrap sampling rate was reduced to 0.50.

Fit Statistics											
Target Name	Data Role	va_d_PARTITIO...	Formatted Partition	Number of Obser...	Average Squared ...	Divisor for ASE	Root Average Squa...	Mean Absolute Er...	Root Mean Absol...	Mean Squared Lo...	Root Mean Squar...
MedianSalePrice	VALIDATE	0	0	1,979	1,123,864,270.0500	1,979	33,524.0849	20,175.5301	142.0406	0.0062	0.0787
MedianSalePrice	TRAIN	1	1	5,935	740,659,945.9370	5,935	22,215.0684	17,119.8144	130.8427	0.0047	0.0687
MedianSalePrice	TEST	2	2	1,979	1,150,401,384.0500	1,979	33,917.5675	20,443.7487	142.9816	0.0064	0.0798

Figure 1.19: Tuned Random Forest Fit Statistics

The tuned Random Forest achieved slightly lower R^2 values compared to the default configuration (0.971 training vs. 0.960 test) and produced more conservative predictions (Figure 1.19). The tuning adjustments increased model bias and reduced model variance, but did not translate into improved predictive accuracy for this dataset. Instead, the

results reflect a typical bias–variance trade-off in tree ensembles, where regularization decreases variance at the expense of reduced fit.

Model Comparison								
Champion	Name	Algorithm Name	Average Squared Error	Data Role	Number of Observations	Root Average Squared Error	Root Mean Absolute Error	Root Mean Squared Logarithm...
•	Forest Default	Forest	833,417,963.7540	TEST	1,979	28,868.9793	132.4296	0.0704
	Forest Tuned	Forest	1,150,401,384.0500	TEST	1,979	33,917.5675	142.9816	0.0798

Figure 1.20: Random Forest Model Comparison

The model comparison indicates that the default Random Forest outperforms the tuned configuration on the test partition. The default model achieves lower ASE, RMSE, and MAE values, suggesting a tighter fit to the observed housing prices. For this dataset, hyperparameter tuning did not improve accuracy, as the baseline configuration already provided an efficient bias–variance balance. Accordingly, the default Random Forest was selected as the champion model for subsequent analysis (Figure 1.20).

Additional tuning procedures such as SAS Viya’s autotune functionality and the Inputs Evaluation macro were considered. However, due to environment-level restrictions and incompatibility with the analytical workspace, these procedures were not executed. Instead, manual hyperparameter tuning was conducted for comparability and model refinement.

1.4.3 Gradient Boosting Models

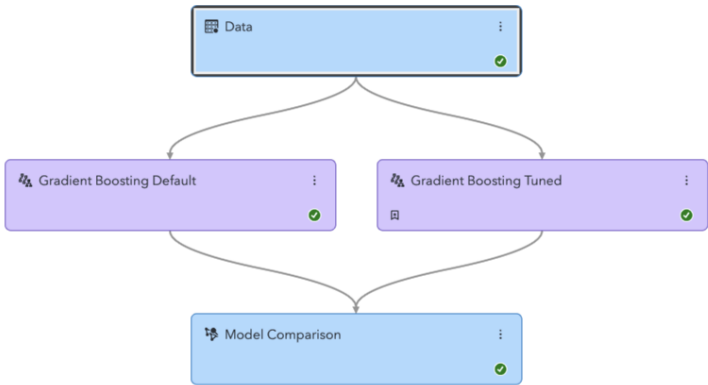


Figure 1.21: Gradient Boosting Pipeline

Gradient boosting was implemented to assess whether a sequential boosting strategy could improve predictive accuracy beyond the bagging-based random forest model (Figure 1.21). Unlike random forests, which reduce variance by aggregating independent trees, gradient boosting builds trees sequentially, each one correcting the residual errors of the previous ensemble. This iterative structure allows the model to capture subtle nonlinearities and interaction effects, which are particularly relevant in heterogeneous housing markets. Both a default configuration and a tuned configuration were trained within the SAS Viya

pipeline, enabling a controlled comparison of boosting performance with and without hyperparameter refinement.

Default Gradient Boosting

A baseline Gradient Boosting model was trained using the default configuration. Under this setup, the ensemble was constructed from 100 boosting iterations with a learning rate of 0.10 and a subsampling rate of 0.50. The default tree-splitting parameters enforced relatively shallow trees (maximum depth of 4, minimum leaf size of 5) and binary splits, producing weak learners in line with the boosting framework. These settings create a conservative baseline that prioritizes bias reduction through sequential residual fitting rather than aggressive variance minimization.

Fit Statistics											
Target Name	Data Role	va_d_PARTITIO...	Formatted Partition	Number of Obser...	Average Squared ...	Divisor for ASE	Root Average Squa...	Mean Absolute Er...	Root Mean Absol...	Mean Squared Lo...	Root Mean Squar...
MedianSalePrice	VALIDATE	0	0	1,979	802,348,599.3140	1,979	28,325.7586	19,017.8830	137.9053	0.0060	0.0773
MedianSalePrice	TRAIN	1	1	5,935	508,966,455.9650	5,935	22,560.2849	16,478.3635	128.3681	0.0049	0.0699
MedianSalePrice	TEST	2	2	1,979	877,132,674.3130	1,979	29,616.4258	19,007.5570	137.8679	0.0059	0.0769

Figure 1.22: Default Gradient Boosting Fit Statistics

Model performance was evaluated using RMSE, MAE, RMSLE and R-squared across the training, validation and test partitions. The default Gradient Boosting configuration achieved an RMSE of approximately 29.6K USD and an MAE of roughly 19.0K USD on the test set, with an R^2 of 0.9692 (Figure 1.22). Performance was consistent across the validation split (R^2 0.9695), indicating stable generalization. The relatively small gap between the training R^2 (0.9803) and test R^2 suggests that the ensemble captured non-linear price dynamics without exhibiting severe overfitting. These results confirm that even the default boosting configuration provides a strong predictive baseline for modeling metropolitan housing prices.

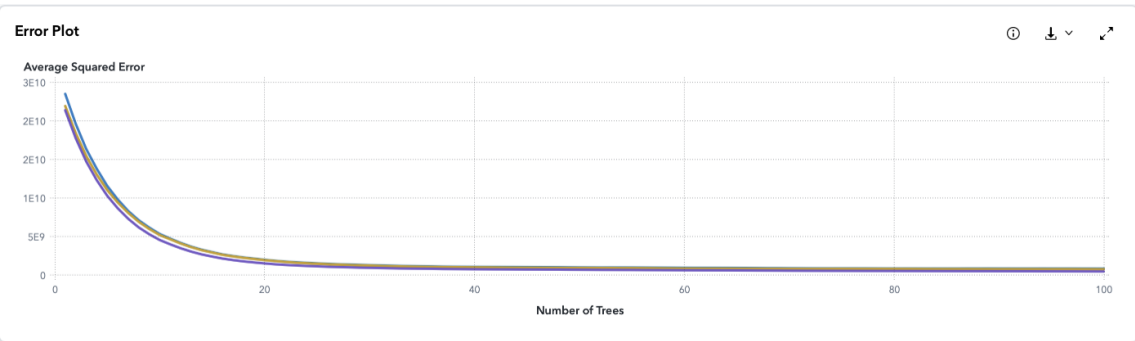


Figure 1.23: Default GB Error Plot

The error declines sharply during the first boosting iterations and then stabilizes, with train, validation and test lines remaining almost identical. This indicates a smooth learning process without visible overfitting under the default settings (Figure 1.23).

Tuned Gradient Boosting

In the tuned configuration, the boosting ensemble was adjusted to improve stability and reduce variance. The number of trees was increased to 300, while the learning rate was reduced to 0.05, allowing the model to learn more gradually. A subsampling rate of 0.60 introduced randomness at each iteration, which helps limit overfitting. L2 regularization was increased to place stronger penalties on complex updates. The tree depth was limited to 6 and the minimum leaf size was increased to 10, which constrains individual trees and reduces sensitivity to noise. The number of interval bins was increased to 200 to allow more detailed splits for continuous predictors. Early stopping was enabled so that training stops when the validation error no longer improves. Overall, the tuning focused on smoother learning behavior and stronger regularization compared to the default configuration.

Fit Statistics											
SAS											
Target Name	Data Role	va_d_PARTITIO...	Formatted Partition	Number of Obser...	Average Squared ...	Divisor for ASE	Root Average Squa...	Mean Absolute Er...	Root Mean Absol...	Mean Squared Lo...	Root Mean Squar...
MedianSalePrice	VALIDATE	0	0	1,979	389,386,999.1020	1,979	19,732.8913	12,959.1960	113.8385	0.0029	0.0542
MedianSalePrice	TRAIN	1	1	5,935	142,710,931.9760	5,935	11,946.1681	8,819.8928	93.9143	0.0016	0.0403
MedianSalePrice	TEST	2	2	1,979	378,742,294.2310	1,979	19,461.3025	12,910.6040	113.6248	0.0030	0.0547

Figure 1.24: Tuned GB Fit Statistics

The tuned Gradient Boosting model delivered strong and stable performance across partitions. On the test partition, the RMSE decreased to approximately 19.5K dollar and the MAE to about 12.9K dollar, while the R^2 increased to 0.9867 (Figure 1.24). The test results were closely aligned with the training (0.9945) and validation (0.9852) scores, suggesting improved generalization without signs of overfitting. Overall, the tuned configuration captured more of the non-linear structure in housing prices.

Model Comparison							
Champion	Name	Algorithm Name	Average Squared Error	Data Role	Number of Observations	Root Average Squared Error	Root Mean Squared Logarithmic Error
★	Gradient Boosting Tuned	Gradient Boosting	378,742,294.2310	TEST	1,979	19,461.3025	113.6248
	Gradient Boosting Default	Gradient Boosting	877,132,674.3130	TEST	1,979	29,616.4258	137.8679

Figure 1.25: Gradient Boosting Model Comparison

The model comparison confirms that the tuned Gradient Boosting configuration consistently outperforms the default model on the test set across all error metrics. The substantial reduction in RMSE and MAE highlights the effectiveness of the tuning choices in improving predictive accuracy while maintaining stable generalization (Figure 1.25).

1.5 Model Comparison

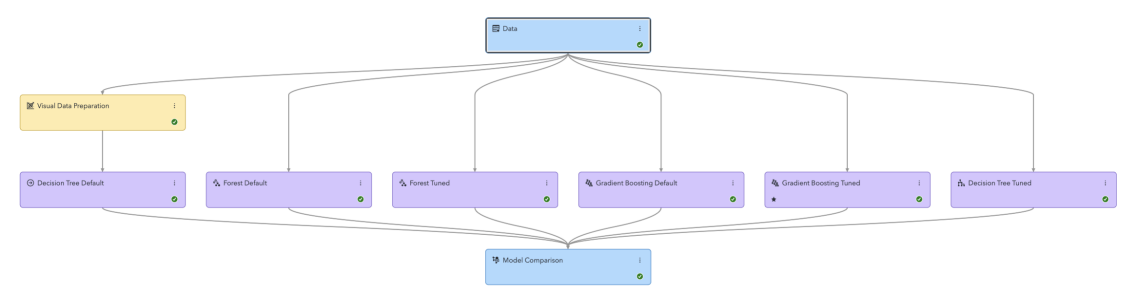


Figure 1.26: Unified Modelling Pipeline

All models were trained and evaluated within a unified SAS Viya pipeline to ensure methodological consistency (Figure 1.26). Using identical data partitions and comparable settings allows performance differences to be attributed to modeling strategy rather than data or preprocessing variation. This design choice enables a fair comparison between single trees, bagging-based ensembles and boosting-based ensembles, and supports the selection of a final best-performing model.

Model Comparison									
Champion	Name	Algorithm Name	T	Average Squared Error	Data Role	Number of Observations	Root Average Squared Error	Root Mean Absolute Error	Root Mean Squared Logarith...
★	Gradient Boosting Tuned	Gradient Boosting		403,394,858.3410	TEST	1,979	20,084.6921	116.5793	0.0572
	Forest Default	Forest		833,417,963.7540	TEST	1,979	28,868.9793	132.4296	0.0704
	Gradient Boosting Default	Gradient Boosting		877,132,674.3130	TEST	1,979	29,616.4258	137.8679	0.0769
	Forest Tuned	Forest		1,150,401,384.0500	TEST	1,979	33,917.5675	142.9816	0.0798
	Decision Tree Tuned	Decision Tree		1,520,934,919.7100	TEST	1,979	38,999.1656	159.4248	0.0999
	Decision Tree Default	Decision Tree		2,022,322,098.2700	TEST	1,979	44,970.2357	178.7069	0.1265

Figure 1.27: Unified Model Comparison

The model comparison on the test partition shows a clear performance ranking across the evaluated approaches. The tuned Gradient Boosting model achieved the lowest error values and was selected as the champion model, with an ASE of 403,394,858, an RMSE of approximately 20.1K, and an MAE of about 13.6K. The default Random Forest and default Gradient Boosting models produced higher errors (RMSE around 28.9K and 29.6K, respectively), while the tuned Random Forest did not improve further (RMSE 33.9K). Both Decision Tree configurations remained the weakest performers (RMSE 39.0K for the tuned tree and 45.0K for the default tree), indicating that a single-tree structure is not sufficient to capture the complex, non-linear relationships in metropolitan housing prices. Overall, the results confirm that boosting with manual tuning provides the most accurate and reliable predictions among the tested models(Figure 1.27).

Fit Statistics																			
Statistic	Train: Gradient Boosting Tuned	Valid: Gradient Boosting Tuned	Test: Gradient Boosting Tuned	Train: Forest Default	Valid: Forest Default	Test: Forest Default	Train: Forest Tuned	Valid: Forest Tuned	Test: Forest Tuned	Train: Decision Tree Default	Valid: Decision Tree Default	Test: Decision Tree Default	Train: Gradient Boosting Default	Valid: Gradient Boosting Default	Test: Gradient Boosting Default	Train: Decision Tree Tuned	Valid: Decision Tree Tuned	Test: Decision Tree Tuned	Train: Gradient Boosting Tuned
Average Squared Error	181,594,763.1830	420,937,290.1940	403,394,583.410	427,948,505.9970	768,472,190.2830	833,417,963.7540	740,659,945.9370	1,123,864,270.0500	1,150,401,384.0500	1,924,656,271.2000	2,205,062,277.1540	2,022,322,098.2700	508,966,455.9650	802,348,599.3140	877,132,674.3130	1,247,636,990.1900	1,401,508,251.8500	1,520,934,919.7100	1,520,934,919.7100
Divisor for ASE	5,935	1,979	1,979	5,935	1,979	1,979	5,935	1,979	1,979	5,935	1,979	1,979	5,935	1,979	1,979	5,935	1,979	1,979	5,935
Formatted Partition	1	0	2	1	0	2	1	0	2	1	0	2	1	0	2	1	0	2	1
Mean Absolute Error	10,018.9762	13,750.6968	13,590.7292	13,005.9719	17,165.9310	17,537.6007	17,119.8144	20,175.5301	20,443.7487	31,651.3756	33,148.5713	31,936.1580	16,478.3635	19,017.8830	19,007.5570	22,777.0931	25,550.2102	25,416.2729	25,416.2729
Mean Squared Logarithmic Error	0.0020	0.0033	0.0033	0.0028	0.0047	0.0050	0.0047	0.0062	0.0064	0.0162	0.0175	0.0160	0.0049	0.0060	0.0059	0.0079	0.0100	0.0100	0.0100
Number of Observations	5,935	1,979	1,979	5,935	1,979	1,979	5,935	1,979	1,979	5,935	1,979	1,979	5,935	1,979	1,979	5,935	1,979	1,979	5,935
R-Square	0.9930	0.9840	0.9858	0.9835	0.9708	0.9707	0.9714	0.9573	0.9596	0.9257	0.9162	0.9290	0.9803	0.9695	0.9692	0.9518	0.9391	0.9466	0.9466
Root Average Squared Error	13,475.7101	20,516.7563	20,084.6921	20,686.9163	27,721.3310	28,868.9793	27,215.0684	33,524.0391	33,917.5675	43,870.9046	46,958.0959	44,970.2357	22,540.2849	28,325.7586	29,616.4258	35,321.9052	40,018.8487	38,999.1656	38,999.1656
Root Mean Absolute Error	100.0948	117.2634	116.5793	114.0437	131.0188	132.4296	130.8427	142.0406	142.9816	177.9083	182.0675	178.7069	128.3681	137.9053	137.8679	150.9208	159.8443	159.4248	159.4248
Root Mean Squared Logarithmic Error	0.0450	0.0572	0.0572	0.0533	0.0684	0.0704	0.0687	0.0787	0.0798	0.1273	0.1322	0.1265	0.0699	0.0773	0.0769	0.0889	0.1000	0.0999	0.0999
Target Name	MedianSalePrice	MedianSalePrice	MedianSalePrice	MedianSalePrice	MedianSalePrice	MedianSalePrice	MedianSalePrice	MedianSalePrice	MedianSalePrice	MedianSalePrice	MedianSalePrice	MedianSalePrice	MedianSalePrice	MedianSalePrice	MedianSalePrice	MedianSalePrice	MedianSalePrice	MedianSalePrice	MedianSalePrice
vs_d_PA	1	0	2	1	0	2	1	0	2	1	0	2	1	0	2	1	0	2	1

Figure 1.28: Unified Fit Statistic

This figure reports the fit statistics of all evaluated models across the training, validation, and test partitions. The tuned Gradient Boosting model consistently achieves the lowest error values, particularly on the test set, with an RMSE of approximately 20.1K and the lowest MAE among all configurations. In contrast, both Decision Tree models exhibit substantially higher error levels, indicating limited capacity to model complex price relationships. Ensemble-based approaches outperform single-tree models, with Gradient Boosting providing the most accurate and stable predictions. The close alignment between training, validation, and test metrics further suggests good generalization performance(Figure 1.28).

Based on test RMSE, the overall performance ranking is: Tuned Gradient Boosting, Default Random Forest, Default Gradient Boosting, Tuned Random Forest, Tuned Decision Tree, and Default Decision Tree. RMSE was used as the main comparison metric since it gives greater weight to large errors, making it more suitable for evaluating housing price predictions.

1.6 Conclusion

This third part of the study examines urban housing price dynamics in U.S. metropolitan areas using visual analytics and tree-based machine learning methods. Rather than focusing solely on prediction, the analysis aims to explain how affordability, rental pressure, and market conditions interact within highly heterogeneous housing markets. By integrating interpretable decision tree models with ensemble learning approaches, the study combines explanatory insight with strong predictive performance.

The analysis followed a structured modeling workflow. A regression decision tree was first used as an interpretable baseline to identify nonlinear relationships and price segmentation patterns. This was followed by Random Forest and Gradient Boosting models to assess the added value of ensemble learning in capturing complex housing market dynamics. All models were implemented within a unified SAS Viya pipeline using identical data partitions, ensuring methodological consistency and fair comparison. Hyperparameter tuning was applied selectively to improve performance while preserving generalization.

Model comparison results reveal a clear performance hierarchy among the evaluated methods. Single decision tree models provided useful market segmentation and interpretability, however, their predictive accuracy remained limited, particularly in higher-priced and more heterogeneous metropolitan areas. Random Forest models improved performance by reducing variance through aggregation, although additional tuning did not consistently lead to further gains. Gradient Boosting models delivered the strongest results, with the tuned specification achieving the lowest prediction errors and stable performance across training, validation, and test sets. Overall, these findings indicate that ensemble approaches, especially boosting, are well suited for modeling the nonlinear and heterogeneous structure of urban housing prices.

Across all modeling approaches, affordability-related variables emerged as the primary drivers of median sale prices. YearsToSave consistently ranked among the most influential features in both tree splits and variable importance measures, highlighting a strong link between long-term affordability constraints and housing prices. MedianRent and new construction sale prices also contributed to price differentiation, particularly in higher-priced market segments. In contrast, short-term market competitiveness indicators, such as sale-to-list ratio and the share of homes sold above list price, played a more limited role, suggesting that long-run affordability factors have a stronger influence on housing prices than temporary bidding pressure.

Taken together, the results provide clear answers to the study’s research questions. Affordability and rental pressure indicators are the most influential factors shaping housing prices across U.S. regions. Tree-based machine learning models outperform traditional linear approaches by effectively capturing nonlinear effects and interaction structures, with ensemble methods delivering substantial gains in predictive accuracy. Moreover, the combination of visual analytics and rule-based interpretation adds practical value by translating complex model behavior into transparent and economically meaningful insights. Overall, this analysis demonstrates that interpretable ensemble modeling, particularly Gradient Boosting, offers a robust and effective framework for analyzing and understanding contemporary urban housing markets.

Bibliography

- [1] Edward B Barbier and Joanne C Burgess. “Sustainable development goal indicators: Analyzing trade-offs and complementarities”. In: *World development* 122 (2019), pp. 295–305.
- [2] Therese Bennich, Nina Weitz, and Henrik Carlsen. “Deciphering the scientific literature on SDG interactions: A review and reading guide”. In: *Science of the Total Environment* 728 (2020), p. 138405.
- [3] Hai-Anh H Dang and Umar Serajuddin. “Tracking the sustainable development goals: Emerging measurement challenges and further reflections”. In: *World Development* 127 (2020), p. 104570.
- [4] Gang-Zhi Fan, Seow Eng Ong, and Hian Chye Koh. “Determinants of house price: A decision tree approach”. In: *Urban Studies* 43.12 (2006), pp. 2301–2315.
- [5] Francesco Fuso Nerini et al. “Mapping synergies and trade-offs between energy and the Sustainable Development Goals”. In: *Nature Energy* 3.1 (2018), pp. 10–15.
- [6] Sergio González et al. “A practical tutorial on bagging and boosting based ensembles for machine learning: Algorithms, software tools, performance study, practical perspectives and opportunities”. In: *Information Fusion* 64 (2020), pp. 205–237.
- [7] DJ Griggs et al. *A guide to SDG interactions: from science to implementation*. International Council for Science, Paris, 2017.
- [8] Tomáš Hák, Svatava Janoušková, and Bedřich Moldan. “Sustainable Development Goals: A need for relevant indicators”. In: *Ecological indicators* 60 (2016), pp. 565–573.
- [9] Sophia-Marie Horvath et al. “Handling a complex agenda: A review and assessment of methods to analyse SDG entity interactions”. In: *Environmental Science & Policy* 131 (2022), pp. 160–176.
- [10] Felix Lorenz et al. “Interpretable machine learning for real estate market analysis”. In: *Real estate economics* 51.5 (2023), pp. 1178–1208.
- [11] Alina Luchian et al. “Harnessing Data Analysis for Global Sustainability: An Integrated Approach to Environmental, Economic, and Social Wellbeing”. In: *3S2024 Final Submission* (2024).
- [12] Alina Luchian et al. “Harnessing Data Analysis for Global Sustainability: An Integrated Approach to Environmental, Economic, and Social Wellbeing”. In: *West, S., Meierhofer, J. Smart Services Summit 2024. Progress in IS*. Springer, 2025.
- [13] David Lusseau and Francesca Mancini. “Income-based variation in Sustainable Development Goal interaction networks”. In: *Nature Sustainability* 2.3 (2019), pp. 242–247.
- [14] Xiao-Li Meng. “What Are the Values of Data, Data Science, or Data Scientists?”. In: (2021).

- [15] David Moher et al. “Preferred reporting items for systematic reviews and meta-analyses: the PRISMA statement”. In: *Annals of internal medicine* 151.4 (2009), pp. 264–269.
- [16] Jonathan D Moyer and David K Bohl. “Alternative pathways to human development: Assessing trade-offs and synergies in achieving the Sustainable Development Goals”. In: *Futures* 105 (2019), pp. 199–210.
- [17] Mehrbakhsh Nilashi et al. “Critical data challenges in measuring the performance of sustainable development goals: Solutions and the role of big-data analytics”. In: *Harvard Data Science Review* 5.3 (2023), pp. 1–36.
- [18] Måns Nilsson et al. “Mapping interactions between the sustainable development goals: lessons learned and ways forward”. In: *Sustainability science* 13 (2018), pp. 1489–1503.
- [19] Prajal Pradhan et al. “A systematic study of sustainable development goal (SDG) interactions”. In: *Earth’s Future* 5.11 (2017), pp. 1169–1179.
- [20] Khairulliza Ahmad Salleh and Lech Janczewski. “Security considerations in big data solutions adoption: Lessons from a case study on a banking institution”. In: *Procedia Computer Science* 164 (2019), pp. 168–176.
- [21] Rafael Sarkis-Onofre et al. “How to properly use the PRISMA Statement”. In: *Systematic Reviews* 10 (2021), pp. 1–3.
- [22] Sharad Saxena. *Tree-Based Machine Learning Methods in SAS Viya*. SAS Institute, 2022.
- [23] Jörn PW Scharlemann et al. “Towards understanding interactions between Sustainable Development Goals: The role of environment–human linkages”. In: *Sustainability science* 15 (2020), pp. 1573–1584.
- [24] David Teh and Tarek Rana. “The use of Internet of Things, Big Data analytics and artificial intelligence for attaining UN’s SDGs”. In: *Handbook of big data and analytics in accounting and auditing*. Springer, 2023, pp. 235–253.
- [25] John Wilder Tukey et al. *Exploratory data analysis*. Vol. 2. Springer, 1977.
- [26] UN. “Follow-up and review of the 2030 Agenda for Sustainable Development at the global level”. In: (2016), p. 4.
- [27] UN. *Global Sustainable Development Report 2019: The Future is Now - Science for Achieving Sustainable Development*. New York, NY, USA: United Nations, 2019. URL: https://sustainabledevelopment.un.org/content/documents/24797GSDR_report_2019.pdf.
- [28] UN. *Transforming Our World: The 2030 Agenda for Sustainable Development. Resolution Adopted by the General Assembly on 25 September 2015, 42809, 1-13*. <https://doi.org/10.1007/s13398-014-0173-7.2>. 2015.